

ASSIGNMENT -11.1

NAME :P.AJAY

HALL.NO:2403A510B4

BATCH:05

Task 1: Stack Implementation

Prompt: Generate a Stack class with push, pop, peek, and is_empty methods.

Code:

```
class Stack:
    """A simple stack implementation using Python list."""

    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item. Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("Pop from empty stack")
        return self.items.pop()

    def peek(self):
        """Return the top item without removing it. Raises IndexError if empty."""
        if self.is_empty():
            raise IndexError("Peek from empty stack")
        return self.items[-1]

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.items) == 0

# Test cases
s = Stack()
s.push(10)
s.push(20)
print("Peek:", s.peek()) # Expected 20
print("Pop:", s.pop()) # Expected 20
print("Is Empty:", s.is_empty()) # Expected False
```

Output:

```
PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Peek: 20
Pop: 20
Is Empty: False
PS C:\Users\AJAY\ai coding>
```

Observation:

(Analysis and AI suggestions for Stack Implementation)

Task 2: Queue Implementation

Prompt: Implement a Queue using Python lists.

Code:

```

class Queue:
    """A simple FIFO queue using Python list."""

    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Insert item at the end of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the front item. Raises IndexError if empty."""
        if self.size() == 0:
            raise IndexError("Dequeue from empty queue")
        return self.items.pop(0)

    def peek(self):
        """Return the front item without removing it."""
        if self.size() == 0:
            raise IndexError("Peek from empty queue")
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

# Test cases
q = Queue()
q.enqueue(1)
q.enqueue(2)
print("Peek:", q.peek())
print("Dequeue:", q.dequeue())
print("Size:", q.size())

```

Output:

```

PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Peek: 1
Dequeue: 1
Size: 1

```

Observation:

(Analysis and AI suggestions for Queue Implementation)

Task 3: Linked List

Prompt: Generate a Singly Linked List with insert and display methods.

Code:

```
class Node:
    """A node in a singly linked list."""

    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """Singly linked list implementation."""

    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node at the end."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def display(self):
        """Return list elements as a Python list."""
        elements, current = [], self.head
        while current:
            elements.append(current.data)
            current = current.next
        return elements

# Test cases
ll = LinkedList()
ll.insert(5)
ll.insert(10)
print("Linked List:", ll.display())
```

Output:

```
Linked List: [5, 10]
PS C:\Users\AJAY\ai coding> C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Linked List: [5, 10]
```

Observation:

(Analysis and AI suggestions for Linked List)

Task 4: Binary Search Tree (BST)

Prompt: Create a BST with insert and in-order traversal methods.

Code:

```
class BSTNode:
    """A node in a Binary Search Tree."""
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BST:
    """Binary Search Tree implementation."""
    def __init__(self):
        self.root = None

    def insert(self, key):
        """Insert a key into the BST."""
        if not self.root:
            self.root = BSTNode(key)
        else:
            self._insert(self.root, key)

    def _insert(self, node, key):
        if key < node.key:
            if node.left:
                self._insert(node.left, key)
            else:
                node.left = BSTNode(key)
        else:
            if node.right:
                self._insert(node.right, key)
            else:
                node.right = BSTNode(key)

    def inorder(self):
        """Return in-order traversal of the BST."""
        result = []
        self._inorder(self.root, result)
        return result

    def _inorder(self, node, result):
        if node:
            self._inorder(node.left, result)
            result.append(node.key)
            self._inorder(node.right, result)

# Test
bst = BST()
for val in [50, 30, 70, 20, 40, 60, 80]:
    bst.insert(val)
```

Output:

```
PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
In-order Traversal: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\AJAY\ai coding>
```

Observation:

(Analysis and AI suggestions for Binary Search Tree (BST))

Task 5: Hash Table

Prompt: Implement a hash table with insert, search, and delete methods with chaining.

Code:

```
class HashTable:
    """Hash table implementation with chaining."""

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert or update key-value pair."""
        idx = self._hash(key)
        for pair in self.table[idx]:
            if pair[0] == key:
                pair[1] = value
                return
        self.table[idx].append([key, value])

    def search(self, key):
        """Search for value by key."""
        idx = self._hash(key)
        for pair in self.table[idx]:
            if pair[0] == key:
                return pair[1]
        return None

    def delete(self, key):
        """Delete key-value pair if exists."""
        idx = self._hash(key)
        for i, pair in enumerate(self.table[idx]):
            if pair[0] == key:
                del self.table[idx][i]
                return True
        return False

# Test
ht = HashTable()
ht.insert("apple", 5)
ht.insert("banana", 7)
print("Search apple:", ht.search("apple"))
ht.delete("apple")
print("Search apple after delete:", ht.search("apple"))
```

Output:

```
PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Search apple: 5
Search apple after delete: None
```

Observation:

(Analysis and AI suggestions for Hash Table)

Task 6: Graph Representation

Prompt: Implement a graph using an adjacency list.

Code:

```
class Graph:
    """Graph represented using adjacency list."""

    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []

    def add_edge(self, v1, v2):
        self.add_vertex(v1)
        self.add_vertex(v2)
        self.adj_list[v1].append(v2)
        self.adj_list[v2].append(v1) # undirected

    def display(self):
        return self.adj_list

# Test
g = Graph()
g.add_edge("A", "B")
g.add_edge("A", "C")
print("Graph:", g.display())
```

Output:

```
PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Graph: {'A': ['B', 'C'], 'B': ['A'], 'C': ['A']}
```

Observation:

(Analysis and AI suggestions for Graph Representation)

Task 7: Priority Queue

Prompt: Implement a priority queue using Python's heapq module.

Code:

```
import heapq

class PriorityQueue:
    """Priority Queue implemented with heapq (min-heap)."""

    def __init__(self):
        self.heap = []

    def enqueue(self, priority, item):
        heapq.heappush(self.heap, (priority, item))

    def dequeue(self):
        if not self.heap:
            raise IndexError("Dequeue from empty priority queue")
        return heapq.heappop(self.heap)[1]

    def display(self):
        return [item for _, item in sorted(self.heap)]

# Test
pq = PriorityQueue()
pq.enqueue(2, "task2")
pq.enqueue(1, "task1")
print("Dequeue:", pq.dequeue())
print("Display:", pq.display())
```

Output:

```
PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Dequeue: task1
Display: ['task2']
PS C:\Users\AJAY\ai coding>
```

Observation:

(Analysis and AI suggestions for Priority Queue)

Task 8: Deque

****Prompt:**** Implement a double-ended queue using collections.deque.

Code:

```
from collections import deque

class DequeDS:
    """Double-ended queue using collections.deque."""

    def __init__(self):
        self.deque = deque()

    def add_front(self, item):
        self.deque.appendleft(item)

    def add_rear(self, item):
        self.deque.append(item)

    def remove_front(self):
        return self.deque.popleft()

    def remove_rear(self):
        return self.deque.pop()

    def display(self):
        return list(self.deque)

# Test
dq = DequeDS()
dq.add_front(1)
dq.add_rear(2)
print("Deque:", dq.display())
dq.remove_front()
print("After remove front:", dq.display())
```

Output:

```
PS C:\Users\AJAY\ai coding> & C:/Users/AJAY/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/AJAY/ai coding/11.1.txt"
Deque: [1, 2]
After remove front: [2]
PS C:\Users\AJAY\ai coding>
```

Observation:

(Analysis and AI suggestions for Deque)

Task 9: Data Structure Comparisons

Prompt: Generate a comparison table of data structures with time complexities.

TABLE:

Data Structure	Insert/Enqueue	Delete/Dequeue	Search/Access	Peek/Traversal	
Stack	O(1)	O(1)	O(n)	O(1)	
Queue	O(1) (rear)	O(1) (front)	O(n)	O(1)	
Linked List	O(1) (head)	O(1) (head)	O(n)	O(n) traversal	
BST (avg)	O(log n)	O(log n)	O(log n)	O(n) traversal	
BST (worst)	O(n)	O(n)	O(n)	O(n) traversal	
Hash Table	O(1) avg	O(1) avg	O(1) avg	N/A	
Graph (Adj List)	O(1) per edge	O(1) per edge	O(V+E)	O(V+E)	
Priority Queue	O(log n)	O(log n)	O(n)	O(1) (min)	
Deque	O(1) ends	O(1) ends	O(n)	O(1)	

Observation:

(Analysis and AI suggestions for Data Structure Comparisons)

Task 10: Real-Time Application Challenge

Prompt: Choose appropriate data structures for given campus system features.

Code:

```
class AttendanceQueue:

    def __init__(self):
        self.records = []

    def log_entry(self, student):
        """Log student entry."""
        self.records.append(student)

    def log_exit(self):
        """Remove the earliest entry (FIFO)."""
        if not self.records:
            raise IndexError("No records to remove")
        return self.records.pop(0)

    def show_records(self):
        return self.records

# Test
att = AttendanceQueue()
att.log_entry("Alice")
att.log_entry("Bob")
print("Records:", att.show_records())
print("Exit:", att.log_exit())
print("Records after exit:", att.show_records())
```

Output:

```
Exit: Alice
Records after exit: ['Bob']
PS C:\Users\AJAY\ai coding>
```

Observation:

(Analysis and AI suggestions for Real-Time Application Challenge)